CHAPTER 28

Response Handling and Stop Reasons

In this chapter

Introduction: Four Chapters of the Same Bug . . . 3

28.1 The Message Object . . . 4

28.2 content Is an Array of Typed Blocks . . . 5

28.3 The Seven Stop Reasons as a Control-Flow Table . . . 6

28.4 The State With No Stop Reason . . . 7

28.5 Continuing a Non-Terminal Turn . . . 8

28.6 usage: Three Numbers, One Response . . . 9

28.7 Streaming and Non-Streaming Must Agree . . . 9

28.8 Visual Analogies . . . 10

28.9 Architecture: The Reader That Assumes Nothing . . . 11

28.10 Examples . . . 11

28.11 Code Walkthrough: The Two Lines That Make It Total . . . 13

28.12 Mini Project: The Response Reader . . . 14

28.13 Panel A — Three Readers, Eighteen Responses . . . 14

28.14 Panel B — The State With No Marker . . . 16

28.15 Panel C — Eight Rejections and One Unknown . . . 16

28.16 Panel D — Three Numbers From One Response . . . 17

28.17 Evaluation and Deployment . . . 18

28.18 Hands-on Labs . . . 18

28.19 Lab Solutions . . . 18

28.20 Best Practices . . . 19

28.21 Common Mistakes . . . 21

28.22 Performance Considerations . . . 23

28.23 Security Considerations . . . 23

28.24 Real Production Examples . . . 24

28.25 Certification Notes . . . 24

28.26 Twenty Practice Questions . . . 25

28.27 Answers and Explanations . . . 26

28.28 Ten Practical Exercises . . . 27

28.29 Five Challenge Labs 27
28.30 Ten Interview Questions 28
28.31 Summary 28
28.32 Glossary 29
28.33 Further Reading 30

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Name every field of the `Message` object and say what each one is for, including why the top-level `model` may not be the model you asked for.
2. Treat `content` as an array of typed blocks, enumerate every block type this book has verified, and handle an unfamiliar one correctly.
3. Branch on all seven `stop_reason` values as a control-flow table, distinguishing the two non-terminal ones and what each of them wants back.
4. Detect a server tool call that has not run, which has **no** `stop_reason` **of its own** and must be found by matching identifiers.
5. Build the follow-up request for a `tool_use` turn and for a `pause_turn` turn, and avoid the four documented 400s that surround them.
6. Explain why `stop_reason` is not an error, why a refusal is an HTTP 200, and why an error can still arrive inside one.
7. Read `usage` correctly — three different token counts from one response — and avoid double-counting thinking tokens.
8. Write a response reader that is a **total function**: correct on everything documented, and loud on everything that is not.

NOTE

Prerequisites

Chapter 16 for the request and for `content[0].text`, the named bug this chapter finally retires. **Chapter 17** for streaming, for where `stop_reason` arrives, and for fixture provenance. **Chapter 18** for thinking blocks and `output_tokens_details`. **Chapter 19** for cache-aware ITPM and for a token's three prices. **Chapter 23** for structured outputs and the `max_tokens` interaction. **Chapter 25** for refusals, `stop_details` and the `fallback` block. **Chapter 26** for the versioning contract that permits new enum values. **Chapter 27** for `max_tokens` as a ceiling on thinking plus text.

Deferred by name: the **tool-use loop as an architecture** is **Chapters 31–35**; **server tools** in detail are **Chapter 34**; **errors, retries and backoff** are **Chapter 29**; **prompt caching economics** are **Chapter 30**; **agent loops** are **Chapters 47–51**.

Introduction: Four Chapters of the Same Bug

Chapter 16 named a bug and, instead of deprecating it, kept it as a function so it could be measured. `response.content[0].text` looks like the obvious way to get the answer out of a response. It is wrong, and every chapter since has found a new way for it to be wrong.

Chapter 16 found that it **raises on three of seven documented** `stop_reason` **values and silently truncates on two more**. Chapter 17 found that a streaming accumulator ignoring block `index` produces `{"city":{"city": "Paris"} "London"}` on parallel tool calls. Chapter 18 found that filtering on `type == "thinking"` round-trips **9 of 12** turns. Chapter 25 found that `content[0].text` **raises on six of seven refusals**, because on a pre-output refusal `content` is not the wrong block — it is the **empty array**.

This chapter puts it out of its misery, and finds something more interesting on the way.

Section 28.16 runs three readers over eighteen response fixtures. The naive reader is correct on **6**, silently wrong on **2**, and raises on **10** — which is bad, but at least mostly loud. The **total reader** this chapter builds is correct on **16** and raises on exactly the **2** fixtures it should: an invented `stop_reason` and an invented block type.

The interesting one is in the middle. The official documentation, in its own best-practice example, recommends this:

```
next((block.text for block in response.content
      if block.type == "text"), "")
```

That is a real improvement on `content[0]`. It skips thinking blocks, it survives an empty `content`, and it never raises. **It never raises** is the finding: across all eighteen fixtures it raised **zero** times and was **silently wrong on three**. It returns `""` for a refusal and `""` for an empty `end_turn` and cannot tell you which. It returns the first of several text blocks and discards the rest. And handed a response with a `stop_reason` your code has never heard of, it cheerfully returns the text as though the response were complete.

A reader that raises has told you something. A reader that returns an empty string has not. That is the whole argument of this chapter, and it is not an argument against the documentation's example — which is offered as a fallback branch in a larger `if`/`elif` — so much as against the way that one line tends to escape the example and become the whole handler.

Two more measurements anchor the chapter. There is a response state with **no `stop_reason` of its own**: a `tool_use` response that also contains a `server_tool_use` block whose result has not arrived. The documentation is unusually direct about it — *"There is no other marker for the state; detect it by checking each `server_tool_use` or `mcp_tool_use` block's `id` for a matching result block."* Of the two fixtures here carrying an unresolved server tool call, a `stop_reason`-only check sees **one**.

And the `usage` object reports three different numbers for one response, only one of which is the bill. A response with 2,000 fresh input tokens, a 16,000-token cache write, a 180,000-token cache read and 8,000 output tokens of which 6,400 are thinking occupies **206,000** context seats, draws **18,000** against ITPM, and costs **$0.40** on Opus 5 — against **$0.21** if you read only `input_tokens` and `output_tokens`, an understatement of **47.50 per cent**. Of the real figure, **40.0 per cent** is thinking nobody will ever read.

28.1 The `Message` Object

Every successful call returns the same shape. Chapter 16 introduced it; here is the whole of it, with what each field is actually for.

Field	Type	What it is for
`id`	string	`msg_...`; the message identifier, distinct from the `request-id` header
`type`	string	always `"message"`
`role`	string	always `"assistant"`

Field	Type	What it is for
`content`	**array**	typed blocks — never a string
`model`	string	**the model that served the request**
`stop_reason`	string	the control-flow branch
`stop_sequence`	string or null	which of your sequences fired
`stop_details`	object or null	**null for every stop reason but `refusal`**
`usage`	object	three different token counts

Two of these are more subtle than they look.

`model` **is the serving model, not the requested one.** Chapter 25 established that under server-side fallback the response's top-level `model` names whichever model actually answered, and a `fallback` content block marks each handoff. If you log the model you *asked for*, your cost attribution and your evaluation results are both attached to the wrong name.

`stop_details` **is null for everything except a refusal.** The stop-reasons documentation states it directly, and it means the field is useless as a discriminator: you cannot test `if response.stop_details` to find out whether something unusual happened, because Chapter 25 already showed it can also be **null on a refusal**, when the decline maps to no named category. Branch on `stop_reason`. Only ever on `stop_reason`.

DEFINITION

Definition — terminal and complete

A response is **terminal** when the assistant turn is over and the API is not waiting for anything: five of the seven stop reasons. It is **complete** when the content is the whole of what Claude meant to say — only `end_turn` and `stop_sequence`. The two are different, and conflating them is how truncated output gets presented as an answer.

28.2 `content` Is an Array of Typed Blocks

The running thread from Chapter 16, restated because this is where it is finally cashed in. Here is every block type this book has verified, and which chapter verified it.

Block type	Appears when	Chapter
`text`	ordinary output	16
`thinking`	reasoning is returned, with a `signature`	18
`redacted_thinking`	reasoning is returned encrypted, with `data`	18
`tool_use`	Claude calls one of **your** tools	31–35
`server_tool_use`	Claude calls a **server-side** tool	34

Block type	Appears when	Chapter
`mcp_tool_use`	Claude calls a tool over MCP	36–40
`web_search_tool_result` and siblings	a server tool's result arrives	34
`fallback`	a server-side fallback handoff	25
`connector_text`	connector output	25

Four rules follow.

There is no guaranteed first block. A thinking block, a `fallback` block or a `tool_use` block can all occupy index 0, which is the mechanical reason `content[0].text` fails so often.

There can be several text blocks. Joining them is the only correct way to reconstruct the answer, and §28.16 measures the documentation's one-liner losing the second of two.

`content` **can be empty.** Chapter 25 found this on refusals; the stop-reasons page documents it for `end_turn` too. An empty array with `end_turn` is a successful, complete response containing nothing.

New block types will appear. Chapter 26 recorded the versioning contract: Anthropic *may "add new variants to enum-like output values."* A reader that ignores an unfamiliar block silently drops content; a reader that raises tells you a feature shipped.

28.3 The Seven Stop Reasons as a Control-Flow Table

Value	Occurs when	What to do	Terminal	Complete
`end_turn`	Claude finished naturally	use the response	yes	**yes**
`max_tokens`	your `max_tokens` was reached	raise the limit or continue	yes	no
`stop_sequence`	one of your sequences fired	read `stop_sequence`	yes	**yes**
`tool_use`	Claude is calling a tool	run it, return `tool_result` blocks	**no**	no
`pause_turn`	a server-tool loop hit its limit	send the assistant content back	**no**	no
`refusal`	Claude declined	read `stop_details`, fall back	yes	no
`model_context_window_exceeded`	the context window filled	treat as truncated	yes	no

The documentation is explicit that these are not errors: *"Unlike errors, which indicate failures in processing your request,* `stop_reason` *tells you why Claude completed its response generation."* Stop reasons live in the body of a **successful** response; errors are 4xx and 5xx with an error body. Chapter 17 added the case that breaks the tidy split — an `error` event can arrive **inside** an HTTP 200 when streaming.

Three of these deserve their own sections.

`max_tokens`

Chapter 27 measured how often this fires when the cap is sized for the answer rather than for thinking plus text: **53.89 per cent** of a workload at `max_tokens` 4,000. The documentation names two consequences. The response text is truncated, and the recommended handling is to **label it** — *"append a notice so the reader knows the output is incomplete"* — or to continue it. And a **tool use block can itself be incomplete**: generation stopped mid-JSON, so the block is there and its input is not trustworthy. **Never execute a tool from a `max_tokens` response without checking.**

`pause_turn`

Returned when the server-side sampling loop reaches its iteration limit while running server tools such as web search. **The default limit is 10 iterations per request.** The response may contain a `server_tool_use` block with no result block. The continuation is to send the response back **as-is**, as an assistant message.

The documentation's own example does something easy to miss: it **replaces** the message list rather than appending to it, *"to maintain alternating roles"*. And it draws a boundary that removes a common confusion outright:

> *"A response that leaves a client `tool_use` block waiting on you never has a `stop_reason` of `pause_turn`: when Claude stops to call your tools, `stop_reason` is `tool_use`."*

So `pause_turn` is exclusively about the **server's** loop, never about yours.

`model_context_window_exceeded`

Chapter 27 established the behaviour: on Claude 4.5 and later the request is accepted and the response carries this stop reason, which *"lets you request the maximum possible tokens without knowing the exact input size."* One further detail belongs here, because it bites at the type level rather than the wire level:

> *"This stop reason is currently typed only in the SDKs' beta namespace."*

You call `client.beta.messages` and use the Beta-prefixed types. On Sonnet 4.5 and newer the API returns the value **without** a beta header; for earlier models you add `model-context-window-exceeded-2025-08-26`. This is Chapter 27's lesson inverted: there, a type existed for a field the server rejected; here, the server returns a value the ordinary type does not name.

28.4 The State With No Stop Reason

A `tool_use` response can contain a `server_tool_use` block whose `id` has no matching result block. That server tool call **has not run**. In the common case Claude has called a server tool and one of your client tools in the same group of parallel calls, and the API returns without running the server tool so that you can run yours first.

> *"There is no other marker for the state; detect it by checking each `server_tool_use` or `mcp_tool_use` block's `id` for a matching result block."*

This is the sharpest instruction in the whole page and it deserves to be read twice. The `stop_reason` is `tool_use`, exactly as it would be with no server tool involved. Nothing in the top-level fields distinguishes the two situations. The state is visible **only** by joining identifiers inside `content`.

Here is the documented shape, transcribed:

```
{
  "stop_reason": "tool_use",
  "content": [
    { "type": "server_tool_use",
      "id": "srvtoolu_01HxbWnMRmbWyMfUtJKC45rA",
      "name": "web_search",
      "input": { "query": "example article" } },
    { "type": "tool_use",
      "id": "toolu_01PjgRJLbXrXEMZwDNYLnBqk",
      "name": "run_command",
      "input": { "command": "uname -a" } }
  ]
}
```

Section 28.17 measures the consequence: of the two fixtures here carrying an unresolved server tool call, a `stop_reason`-only check sees **one of two**.

There is a second reading of the same shape, and the documentation flags it. Under **programmatic tool calling**, a client `tool_use` block can come from code running inside the `code_execution` tool rather than from Claude directly, and its `caller` field names the `code_execution` block that called it. That code has already started and is paused waiting for your `tool_result` blocks; sending them **resumes** the execution rather than starting a deferred tool. The follow-up user message is the same in both cases, with one addition: pass back the `id` from the response's `container` field. Programmatic tool calling in full is Chapter 35.

28.5 Continuing a Non-Terminal Turn

The two non-terminal stop reasons want opposite things, and the documentation attaches four distinct 400s to the near misses.

For `tool_use`, the continuation is a user message of `tool_result` blocks, one for every `tool_use` block, with two extra rules: *"that message must contain nothing except the `tool_result` blocks, and the request must keep the same `tools` array."*

What you did	What you get
left out a `tool_result`	`` `tool_use ids were found without tool_result blocks immediately after` ``
put a `tool_result` after other content	the same error, *"earlier"*
appended text after the `tool_result` blocks, with a server tool waiting	`` ` web_search `tool use with id` srvtoolu_… `was found without a corresponding` web_search_tool_result `block` ` ``
dropped the waiting server tool from `tools`	a 400 whose message ends `` `but no` web_search `tool was provided` ``

The mechanism behind the third row is worth understanding rather than memorising. *"Adding anything after the `tool_result` blocks in that user message, such as text, ends the assistant turn"* — and once the turn has ended, the server tool that was waiting can never produce its result block, so the request is

invalid for that reason rather than for the text. The documented way to give Claude more input is *"a separate user message after the turn completes."*

When the continuation is valid, the API *"attaches your results to the still-open assistant turn, runs the deferred server tool... and continues the turn"*, and the next response's `content` **starts with the result block** answering the previous response's `server_tool_use` id.

For `pause_turn`, the continuation is the assistant content, unchanged. Sending `tool_result` blocks instead is wrong, because there were no client `tool_use` blocks to answer.

> WARNING
>
> **A tension this book cannot resolve**
>
> The documented `pause_turn` continuation ends the `messages` array with an **assistant** message. Chapter 16 recorded that prefilling an assistant message is unsupported on Claude 4.6 and later, with a 400 whose message reads *"The conversation must end with a user message."* The documentation does not state how these two rules interact — whether continuing a paused turn is exempt, or is not considered a prefill. This book records it as **UNKNOWN**, and §28.18's validator raises rather than guessing. If you rely on it, verify it against your model and report what you find.

28.6 `usage`: Three Numbers, One Response

Chapter 19 established that a token has three prices — a seat in the context window, a draw on a rate-limit bucket, and a line on the invoice — and that these are three different quantities. The `usage` object reports all of them, in fields that are easy to add up incorrectly.

Field	Seat?	ITPM?	Billed at
`input_tokens`	yes	**yes**	1× input
`cache_creation_input_tokens`	yes	**yes**	1.25× (5-minute write)
`cache_read_input_tokens`	yes	**no**	0.1×
`output_tokens`	yes	OTPM	5× input on every current model
`output_tokens_details.thinking_tokens`	—	—	**a split of `output_tokens`, not an addition**
`usage.iterations`	—	—	every attempt under server-side fallback (Chapter 25)

Two traps, both measured in §28.19.

Cache reads are free of ITPM and not free of money. They occupy a context seat and are billed at a tenth of the input rate, but draw nothing from the input-tokens-per-minute bucket. A capacity plan built from context seats will be wrong by however much of your prefix is cached.

Thinking tokens are already inside `output_tokens`. Adding them is double-counting; in §28.19 it inflates the context accounting by 6,400 tokens on a single response.

28.7 Streaming and Non-Streaming Must Agree

Chapter 17 built an SSE parser whose reassembled output was asserted **exactly equal** to hand-written non-streaming payloads on all six fixtures. The stop-reasons page adds the specific rule for this field:

> *"When using streaming,* `stop_reason` *is:* `null` *in the initial* `message_start` *event; provided in the* `message_delta` *event; not provided in any other events."*

So a streaming client learns the branch **last**, after it has already rendered text to a user. Chapter 17's running rule is the resolution: **render freely, act only after** `message_stop`**.** A refusal, a truncation and a tool call all look like ordinary text until the `message_delta` arrives.

And the SDKs offer a third path worth knowing: `client.messages.stream(...)` followed by `stream.get_final_message()` consumes the stream for you and returns *"the complete* `Message` *object, identical to what a non-streaming call returns"* — which is the documented statement of the equivalence Chapter 17 measured.

28.8 Visual Analogies

VISUAL ANALOGY

A parcel with a delivery note

The `Message` object is a parcel. `content` is what is inside — several items, of different kinds, in no guaranteed order. `stop_reason` is the delivery note taped to the outside, and it is the only thing that tells you whether the parcel is everything you ordered, part of it, or a note saying the order was refused.

Where it breaks. A real parcel's contents are self-describing; you can tell a book from a brick. Here, a truncated answer and a complete one look identical inside the box — the same block type, the same field, prose that stops at a plausible place. **Only the note distinguishes them**, which is why reading the contents and ignoring the note is the specific failure this chapter is about.

VISUAL ANALOGY

A conversation that has not finished its sentence

The two non-terminal stop reasons are someone stopping mid-sentence for two different reasons. `tool_use` is *"and what was the figure?"* — they are waiting for you to answer. `pause_turn` is *"give me a moment"* — they are waiting for themselves, and the only thing you can helpfully do is nod.

Where it breaks. In conversation, answering when someone wanted a moment is merely awkward. Here it is a 400, and so is nodding when they asked you a question. And the analogy has no equivalent of the third case: an unresolved server tool call, where they have asked you a question **and** are waiting for themselves, and nothing in their tone tells you.

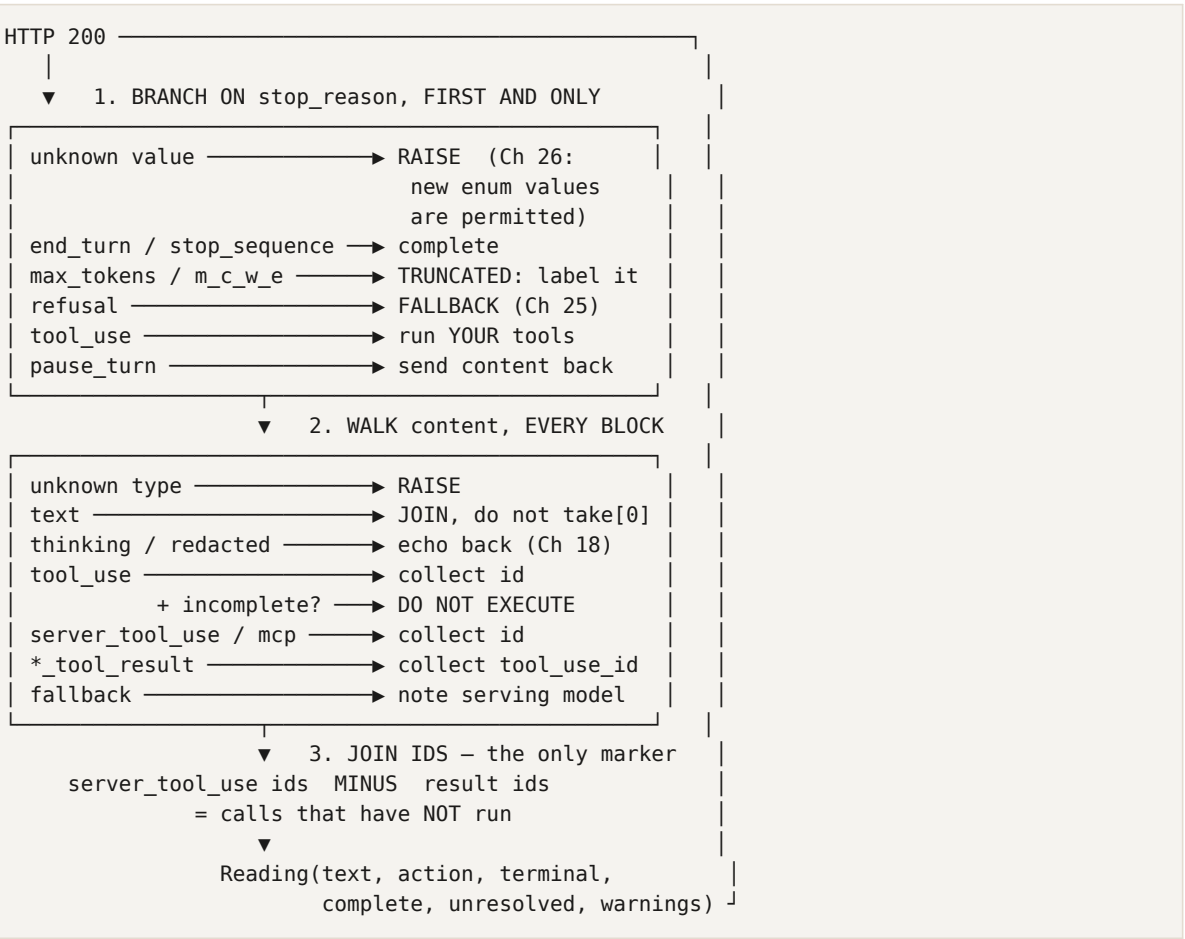
VISUAL ANALOGY

Three meters on one supply

`usage` is a building with three meters: floor space occupied, peak demand, and the bill. A cached prefix is a room you have already furnished — it takes up floor space and appears on the bill at a tenth of the rate, but it draws no peak demand at all.

Where it breaks. Meters measure the same physical thing in different units. These do not: a context seat is a hard capacity limit, an ITPM draw is a rate over a sliding minute, and the bill is a monthly total. Converting between them is not a matter of units, and the naive conversion — read `input_tokens` and `output_tokens`, multiply by the price — understates the bill by **47.50 per cent** in §28.19.

28.9 Architecture: The Reader That Assumes Nothing



Three rules the picture encodes. `stop_reason` **is read before anything else**, because it determines whether the content means what it appears to mean. **Every block is visited**, because the answer may be spread across several and because an unfamiliar one is news. And **the unresolved-server-tool state is computed, not read**, because it is not reported anywhere.

28.10 Examples

Example 1 — the branch, in the documentation's own shape. Note the final `else`, which is where the one-liner lives and where §28.16 finds the problem:

```
def handle_response(response):
    if response.stop_reason == "tool_use":
        return handle_tool_use(response)
    elif response.stop_reason == "max_tokens":
        return handle_truncation(response)
    elif response.stop_reason == "model_context_window_exceeded":
        return handle_context_limit(response)
    elif response.stop_reason == "pause_turn":
        return handle_pause(response)
    elif response.stop_reason == "refusal":
        return handle_refusal(response)
    else:
        return next((b.text for b in response.content
                     if b.type == "text"), "")
```

Example 2 — labelling a truncated answer, which the documentation recommends over presenting it whole:

```
if response.stop_reason == "max_tokens":
    note = "[Response truncated due to max_tokens limit]"
elif response.stop_reason == "model_context_window_exceeded":
    note = "[Response truncated due to context window limit]"
```

Example 3 — the `pause_turn` loop, with the message list **replaced** rather than appended to:

```
for _ in range(max_continuations):
    response = client.messages.create(model="claude-opus-5",
                                      max_tokens=4096,
                                      messages=messages, tools=tools)
    if response.stop_reason != "pause_turn":
        return response
    messages = [{"role": "user", "content": user_query},
                {"role": "assistant", "content": response.content}]
```

Example 4 — the continuation for a mixed response. Nothing but `tool_result` blocks, and the same `tools` array:

```
{ "role": "user",
  "content": [
    { "type": "tool_result",
      "tool_use_id": "toolu_01PjgRJLbXrXEMZwDNYLnBqk",
      "content": "Linux demo-host 6.8.0-52-generic x86_64 GNU/Linux" }
  ] }
```

Example 5 — finding the state nothing reports.

```
server_ids = [b["id"] for b in response["content"]
              if b["type"] in ("server_tool_use", "mcp_tool_use")]
result_ids = {b.get("tool_use_id") for b in response["content"]}
unresolved = [i for i in server_ids if i not in result_ids]
```

Example 6 — reading `usage` without double-counting.

```
u = response["usage"]
seats = (u["input_tokens"] + u.get("cache_creation_input_tokens", 0)
         + u.get("cache_read_input_tokens", 0) + u["output_tokens"])
itpm = u["input_tokens"] + u.get("cache_creation_input_tokens", 0)
thinking = u.get("output_tokens_details", {}).get("thinking_tokens", 0)
assert thinking <= u["output_tokens"]     # a split, not an addition
```

28.11 Code Walkthrough: The Two Lines That Make It Total

`reader.py` is ninety lines and two of them carry the argument.

```
    stop = response.get("stop_reason")
    if stop not in STOP_REASONS:
        raise UnknownStopReason(
            f"stop_reason {stop!r} is not one of "
            f"{sorted(STOP_REASONS)}; the API may add new values, so "
            f"this must be handled explicitly")
```

Line 2 is a membership test against a table of seven, and line 3 raises. That is the whole difference between this reader and the documentation's. The justification is on line 5 and comes from Chapter 26: the versioning contract preserves your existing fields and explicitly reserves the right to *"add new variants to enum-like output values"*. A handler with an `else` branch is a handler that will one day treat a new stop reason as `end_turn`.

```
        kind = block.get("type")
        if kind not in KNOWN_BLOCKS:
            raise UnknownBlockType(
                f"content[{i}] has type {kind!r}, which this reader "
                f"has not verified")
```

The same discipline one level down, and for the same reason. A reader that skips an unfamiliar block quietly drops content; this one reports that a feature has shipped.

```
        if kind in TEXT_BLOCKS:
            parts.append(block["text"])
```

`append`, not `return`. One line, and it is the difference between the documentation's `next(...)` — which takes the first text block and discards the rest — and the correct answer. $28.16 measures it costing the second half of a two-block response.

```
    unresolved = [i for i in server_tools if i not in results]
```

The state with no marker, computed. `server_tools` accumulates every `server_tool_use` and `mcp_tool_use` id; `results` accumulates every `tool_use_id` seen on a result block. The set difference is exactly the documented detection rule, and there is nowhere else to get it from.

```
    complete = stop in ("end_turn", "stop_sequence")
```

Two of seven. Everything else is either unfinished or cut short, and keeping `complete` separate from `terminal` is what stops a truncated answer being rendered as a whole one.

28.12 Mini Project: The Response Reader

What it is. A total reader, a continuation validator and a usage accountant — the three things you need between an HTTP 200 and an effect.

```
ch28/
  wire.py       copied from ch16/ — the request side
  params.py     copied from ch27/ — predicts the stop reason
  fixtures.py   18 responses, 12 DOC and 6 BUILT
  reader.py     the total reader: raises on the unknown
  compare.py    three readers scored over the same corpus
  turns.py      the continuation validator, 8 documented 400s
  usage.py      three counts from one usage object
  report.py     prints every figure in this chapter
  test_ch28.py  30 tests
```

```
   fixtures ──┬─▶ naive_first   content[0].text
              ├─▶ doc_next      next((b.text ...), "")
              └─▶ read          the total reader
                        │
                        ▼   correct / silently wrong / raised
                    compare.tally()
   response + your results ──▶ turns.continuation ──┬─▶ Rejected(400)
                                                    ├─▶ Unverified
                                                    └─▶ what will run
   usage object ──▶ usage.account ──┬─▶ context seats
                                    ├─▶ ITPM / OTPM draw
                                    └─▶ dollars, and the naive error
```

```
cd ch28 && python3 report.py        # every figure below
cd ch28 && python3 -m pytest -q     # 30 passed
```

Deployment. The reader belongs in the same inbound middleware Chapter 26 described, so that no call site ever touches `content` directly. The continuation validator belongs in your agent loop, next to Chapter 27's `params.check` — between them, the request and the response are both validated offline.

28.13 Panel A — Three Readers, Eighteen Responses

Eighteen fixtures: **12 DOC**, transcribed from the documentation, and **6 BUILT** to exercise a documented contract.

Fixture	Prov.	`content[0].text`	doc one-liner	total
`end_turn`, one text block	DOC	correct	correct	correct
`end_turn`, **empty content**	DOC	**raised**	correct	correct

Fixture	Prov.	`content[0].text`	doc one-liner	total
`end_turn`, **two text blocks**	BUILT	**wrong**	**wrong**	correct
`end_turn`, thinking then text	BUILT	**raised**	correct	correct
`end_turn`, redacted_thinking then text	BUILT	**raised**	correct	correct
`max_tokens`, truncated text	DOC	correct	correct	correct
`max_tokens`, **incomplete tool_use block**	DOC	correct	correct	correct
`stop_sequence`	DOC	correct	correct	correct
`tool_use`, one client tool	DOC	**raised**	correct	correct
`tool_use`, **mixed with an unresolved server tool**	DOC	**raised**	correct	correct
`tool_use`, thinking block first	BUILT	**raised**	correct	correct
`pause_turn`, unresolved server tool	DOC	correct	correct	correct
`refusal`, empty content, category `cyber`	DOC	**raised**	correct	correct
`refusal`, **null** `stop_details`	DOC	**raised**	correct	correct
`model_context_window_exceeded`	DOC	correct	correct	correct
a `fallback` handoff, then text	DOC	**raised**	correct	correct
an unknown `stop_reason`	BUILT	**wrong**	**wrong**	**raised**
an unknown block type	BUILT	**raised**	**wrong**	**raised**

Reader	correct	silently wrong	raised
`content[0].text`	6	2	**10**
the documentation's one-liner	15	**3**	**0**
the total reader	**16**	**0**	**2**

Read the middle row carefully, because it is the finding. The one-liner is a genuine improvement — it skips thinking blocks, survives an empty `content`, and is correct on fifteen fixtures. **It raises zero times**, on any input, ever. That is what a total function does, and it means the three fixtures where it is wrong are wrong in complete silence:

- **two text blocks**: it returns `"First part. "` and discards `"Second part."`
- **an unknown** `stop_reason`: it returns the text as though the response were complete
- **an unknown block type**: it returns `""`, indistinguishable from a refusal

The total reader is correct on sixteen and raises on exactly the two it should. Its `correct` count is not higher than fifteen by accident: the extra one is the two-text-block case, and joining is a one-word change.

KEY TAKEAWAYS

The finding in one line

A reader that raises has told you something; a reader that returns `""` has not. The documentation's one-liner is safer than `content[0].text` and less safe than it looks, because it converts three different situations — refusal, empty answer, unrecognised content — into the same empty string.

28.14 Panel B — The State With No Marker

Two of the eighteen fixtures carry a `server_tool_use` block with no matching result. A check that reads only `stop_reason` sees **one of two**: the `pause_turn` one, because that stop reason is itself unusual enough to investigate. The `tool_use` one is invisible — it is the ordinary tool-calling shape, and the server tool is simply sitting in the content, unrun.

Detection method	finds
`stop_reason == "pause_turn"`	**1 of 2**
matching `server_tool_use` ids against result ids	**2 of 2**

This is one of the few places the documentation states the absence of a signal in so many words — *"There is no other marker for the state"* — and it is worth treating that sentence as a specification. Anything you want to know about an unresolved server tool must be derived from `content`.

The consequence for an agent loop is concrete. When you return your client `tool_result` blocks, the API *"runs the deferred server tool… and continues the turn"*, and the **next** response's `content` starts with the result block answering the earlier `server_tool_use` id. A loop that does not know this will see a result block for a call it does not remember making.

28.15 Panel C — Eight Rejections and One Unknown

#	Case	Prov.	Result
1	a `tool_result` missing entirely	DOC	400
2	a `tool_result` placed after other content	DOC	400
3	text appended after the `tool_result` blocks, server tool waiting	DOC	400
4	text appended with no server tool waiting	DOC	400
5	the `tools` array no longer defines the waiting server tool	DOC	400
6	echoing the assistant content for a `tool_use` response	DOC	400
7	`tool_result` blocks sent for a `pause_turn` response	DOC	400
8	continuing a **terminal** response at all	BUILT	400

Rejected 8 of 8, offline. Accepted 1 of 1. And one case raised as UNKNOWN:

Case	Why
the `pause_turn` continuation ends the array on an **assistant** message	the prefill error states *"The conversation must end with a user message"* on Claude 4.6 and later; the interaction is documented neither way

Rows 1 and 3 are worth separating because they produce **different** errors from what looks like the same mistake. A missing `tool_result` fails with `` `tool_use ids were found without tool_result blocks immediately after. Text appended *after* correct tool_result blocks ends the assistant turn, which strands the waiting server tool, and the error names it instead: web_search tool use with id srvtoolu_… was found without a corresponding web_search_tool_result block `. Same intention, same-looking message, two different diagnoses — and the second one mentions a tool you did not call.

Row 6 is the one that catches people migrating from a `pause_turn` loop. Echoing `response.content` back as an assistant message is the correct continuation for `pause_turn` and is an **assistant prefill** for `tool_use` — unsupported on Claude 4.6 and later.

28.16 Panel D — Three Numbers From One Response

A single response with a large cached prefix and adaptive thinking:

```
{"input_tokens": 2000, "cache_creation_input_tokens": 16000,
 "cache_read_input_tokens": 180000, "output_tokens": 8000,
 "output_tokens_details": {"thinking_tokens": 6400}}
```

Quantity	Value
context seats occupied	**206,000**
ITPM draw (cache-aware)	**18,000**
OTPM draw	8,000
billed, correctly	**$0.4000**
billed, reading only `input_tokens` + `output_tokens`	$0.2100
understatement	**47.50%**
of the real bill, thinking	$0.1600 — **40.0%**
context seats if thinking is **added** to output	212,400 (wrong by 6,400)

Four readings.

The context seat and the rate-limit draw differ by 11.4× on this response. A capacity plan built on either number alone is wrong; Chapter 19 built the plan, and this is the response-side view of the same arithmetic.

Reading two fields understates the bill by nearly half. The missing money is the cache: 16,000 tokens written at 1.25× and 180,000 read at 0.1× come to $0.19, and neither appears in `input_tokens`.

Two-fifths of the bill is reasoning nobody will ever read. At Opus 5's $25 per MTok output rate, 6,400 thinking tokens cost $0.16 of a $0.40 response. Chapter 18 measured what carrying that thinking forward does to a conversation; this is what producing it costs once.

`thinking_tokens` **is a split, not an addition.** Adding it to `output_tokens` inflates the seat count by exactly 6,400 here, and the error compounds across a conversation. The assertion `thinking_tokens <= output_tokens` costs one line and catches it.

28.17 Evaluation and Deployment

Measure	Value
Fixtures	18 — **12 DOC**, 6 BUILT
Stop reasons handled	7 of 7, with 2 non-terminal
Block types handled	9 families
Documented continuation rejections reproduced	8 of 8
UNKNOWN cases raised	1 of 1
Readers compared	3
Tests	30, passing, no key, no network

Deploy the reader as the **only** thing that touches `content`. If a call site can reach `response["content"]`, it can reach `content[0]`, and this chapter is a record of what happens next. The continuation validator belongs inside the agent loop; Chapter 27's `params.check` belongs on the way out; Chapter 26's `preflight` belongs above both.

28.18 Hands-on Labs

Lab 28.1 — Build the branch. Write a function taking a response and returning one of seven actions, raising on an eighth. *Success criterion:* an invented `stop_reason` raises, and your test asserts the exception type rather than merely that something failed.

Lab 28.2 — Join, do not index. Write a text extractor and test it against a response with two text blocks and a leading thinking block. *Success criterion:* you get both halves in order.

Lab 28.3 — Find the unresolved server tool. Given the mixed `tool_use` fixture, report which server tool calls have not run. *Success criterion:* you find it from `content` alone, without reading `stop_reason`.

Lab 28.4 — Reproduce three continuation 400s. Pick rows 1, 3 and 5 from Panel C. *Success criterion:* your three errors have three different messages, and you can say why.

Lab 28.5 — Account for one response. Compute context seats, ITPM draw and dollars from a real `usage` object. *Success criterion:* your three numbers differ from each other, and you can explain each difference in one sentence.

28.19 Lab Solutions

28.1. A dictionary from `stop_reason` to `(action, terminal)`, a membership test, and a raise. Keep `terminal` and `complete` as separate booleans — five stop reasons are terminal and only two are complete, and collapsing them is how truncated output gets presented as an answer. See `reader.read`.

28.2. Accumulate into a list and join. The whole fix is `append` instead of `return`. `test_text_is_joined_across_all_text_blocks` asserts both that the total reader gets `"First part. Second part."` and that the documentation's one-liner gets only the first half — the failure is worth keeping in the suite, not just the fix.

28.3. Collect every `server_tool_use` and `mcp_tool_use` id, collect every `tool_use_id` on a result block, take the difference. There is no field to read; the documentation says so explicitly. See `reader.read` and `compare.action_coverage`.

28.4. Row 1 is a missing result and fails on the `tool_result` rule. Row 3 is a *complete* set of results followed by text, which ends the assistant turn and strands the waiting server tool — so the error names the server tool, not your results. Row 5 is a valid continuation whose `tools` array no longer defines the tool the API is about to run. Three different rules, three different messages. See `turns.CASES`.

28.5. Seats are the sum of all four token fields. ITPM is `input_tokens + cache_creation_input_tokens` and **excludes reads**. Dollars apply 1×, 1.25×, 0.1× and 5× respectively. The three numbers here are 206,000, 18,000 and $0.40, and the first two differ by 11.4× because 180,000 cached tokens occupy the window and draw no rate limit. See `usage.account`.

28.20 Best Practices

BEST PRACTICE

Branch on `stop_reason` before you look at `content`

It is the only field that tells you what the content means. `stop_details` cannot do the job — it is null for six of seven stop reasons and can be null on the seventh.

BEST PRACTICE

Raise on an unknown `stop_reason` and an unknown block type

Chapter 26's versioning contract permits new enum values. An `else` branch will silently treat the future as `end_turn`.

BEST PRACTICE

Join every text block; never index into `content`

There is no guaranteed first block, there may be several text blocks, and `content` may be empty. One reader, one place, no call site touching the array.

BEST PRACTICE

Keep `terminal` and `complete` as separate facts

Five stop reasons are terminal; only `end_turn` and `stop_sequence` are complete. Label a truncated answer, as the documentation recommends, or continue it — but never present it as whole.

BEST PRACTICE

Never execute a tool from a `max_tokens` response without checking

The documentation names incomplete tool use blocks as a consequence of truncation. The block exists; its input was cut off mid-generation.

BEST PRACTICE

Detect unresolved server tool calls by identifier

There is no marker. Match every `server_tool_use` and `mcp_tool_use` id against the result blocks, and expect the result to arrive in the **next** response.

BEST PRACTICE

Send back exactly what each non-terminal stop reason wants

`tool_use` wants a user message of nothing but `tool_result` blocks, with the same `tools` array. `pause_turn` wants the assistant content, unchanged, with the message list **replaced** to keep roles alternating.

BEST PRACTICE

Give Claude extra input as a separate user message

Appending text to the `tool_result` message ends the assistant turn and produces a 400 that names a tool you did not call.

BEST PRACTICE

Log the response's `model`, not the one you requested

Under server-side fallback they differ, and a `fallback` block marks the handoff. Cost attribution and evaluation results both depend on the serving model.

BEST PRACTICE

Read `usage` as three numbers, and assert the invariant

Seats, ITPM draw and dollars are different quantities. `thinking_tokens <= output_tokens` is one line and catches the commonest double-count.

BEST PRACTICE

Under streaming, render freely and act only after `message_stop`

`stop_reason` is `null` in `message_start` and arrives in `message_delta`. Everything before that is provisional.

BEST PRACTICE

Use the SDK's `get_final_message()` when you do not need incremental events

It consumes the stream and returns a `Message` *"identical to what a non-streaming call returns"* — the documented form of the equivalence Chapter 17 measured.

28.21 Common Mistakes

COMMON MISTAKE

`response.content[0].text`

Symptom. An `IndexError`, a `KeyError`, or a thinking block rendered to a user. **Cause.** There is no guaranteed first block, and `content` can be empty. **Fix.** One reader that joins text blocks and branches on `stop_reason` first. **Detection.** Panel A: correct on 6 of 18, raising on 10.

COMMON MISTAKE

The one-liner that never raises

Symptom. Empty answers in production with no error, no log line and no pattern. **Cause.** `next((b.text for b in content if b.type == "text"), "")` returns `""` for a refusal, for an empty `end_turn`, and for a response made entirely of block types you do not recognise. **Fix.** Return a reading, not a string. Make the empty cases distinguishable. **Detection.** Panel A: silently wrong on 3 of 18, raising on 0.

COMMON MISTAKE

Treating a stop reason as an error

Symptom. Retry storms on refusals, or refusals invisible to monitoring. **Cause.** A stop reason is part of a **successful** response; errors are 4xx and 5xx. **Fix.** Two separate code paths, and a refusal metric that is not an error metric. **Detection.** Chapter 25 measured an error-rate monitor seeing 0 of 6 refusals.

COMMON MISTAKE

Presenting a truncated response as complete

Symptom. Answers that stop mid-sentence, reported by users rather than by monitoring. **Cause.** `max_tokens` and `model_context_window_exceeded` are terminal but not complete. **Fix.** Label or continue. The documentation supplies both patterns. **Detection.** A counter on each stop reason; Chapter 27 measured 53.89 per cent truncation at a badly sized cap.

COMMON MISTAKE

Executing a truncated tool call

Symptom. A tool runs with empty or malformed arguments after a long response. **Cause.** `max_tokens` can cut a `tool_use` block mid-JSON. **Fix.** Refuse to execute tools from a `max_tokens` response. **Detection.** Assert that the response is not truncated before dispatching.

COMMON MISTAKE

Missing an unresolved server tool call

Symptom. A result block arrives in a later response for a call your loop has no record of. **Cause.** A `tool_use` response can carry an unrun `server_tool_use` block, and nothing at the top level says so. **Fix.** Compute the set difference over ids. **Detection.** Panel B: `stop_reason` alone finds 1 of 2.

COMMON MISTAKE

Appending text to the `tool_result` message

Symptom. A 400 naming a `web_search` tool you never called. **Cause.** Anything after the `tool_result` blocks ends the assistant turn, stranding the waiting server tool. **Fix.** `tool_result` blocks only; extra input goes in a later user message. **Detection.** Panel C, rows 3 and 4.

COMMON MISTAKE

Changing the `tools` array on the continuation

Symptom. A 400 ending `but no web_search tool was provided. **Cause.** The API is about to run a deferred server tool and the request no longer defines it. **Fix.** Keep the same tools` array for the whole turn. **Detection.** Panel C, row 5.

COMMON MISTAKE

Confusing the two non-terminal continuations

Symptom. A 400 on a loop that worked before server tools were added. **Cause.** `tool_use` wants `tool_result` blocks; `pause_turn` wants the assistant content back. Doing either for the other is an error — and echoing content for `tool_use` is an assistant prefill, unsupported on 4.6 and later. **Fix.** Branch on `stop_reason` and keep the two paths apart. **Detection.** Panel C, rows 6 and 7.

COMMON MISTAKE

Appending to the message list on `pause_turn`

Symptom. A 400 about message roles after several continuations. **Cause.** The documented recipe **replaces** the list *"to maintain alternating roles"*. **Fix.** Rebuild the two-message list each iteration. **Detection.** Assert alternating roles before every request.

COMMON MISTAKE

Adding `thinking_tokens` to `output_tokens`

Symptom. Context accounting that drifts steadily high over a conversation. **Cause.** It is a split of `output_tokens`, not an addition to it. **Fix.** `assert thinking_tokens <= output_tokens`. **Detection.** Panel D: 212,400 seats instead of 206,000, on one response.

COMMON MISTAKE

Costing a response from `input_tokens` and `output_tokens`

Symptom. A cost model that consistently understates the invoice. **Cause.** Cache writes bill at 1.25× and reads at 0.1×, and neither appears in `input_tokens`. **Fix.** Include all four token fields with their multipliers. **Detection.** Panel D: **47.50 per cent** understated on one response.

28.22 Performance Considerations

Reading a response is free; reading it twice is not. The reader here is a single pass over `content` that computes text, tool ids, unresolved server calls and warnings together. The tempting alternative — one comprehension per question — turns an O(n) walk into four, which matters when `content` carries a hundred blocks from a long tool-using turn.

The stop reason determines your latency profile, not the reverse. A `tool_use` or `pause_turn` response means at least one more round trip, and Chapter 11's two-term model applies to each: every continuation pays prefill on the whole accumulated conversation. An agent loop's wall clock is the sum of its turns, and its token bill is quadratic in turns for the reason Chapter 16 measured at **26.2×** over thirty turns.

`pause_turn` **has a hard floor of ten server-tool iterations per request.** A task needing more will take more requests, and each is a fresh prefill of the whole turn.

The cached prefix is what keeps that affordable. Panel D's response draws 18,000 ITPM against 206,000 context seats precisely because 180,000 tokens were read from cache. Chapter 30 makes that a design tool rather than an observation.

Streaming changes when you learn the branch, not how fast it arrives. `stop_reason` comes in `message_delta`, near the end. Rendering earlier is a perception improvement, and Chapter 17's rule stands: act only after `message_stop`.

28.23 Security Considerations

A truncated tool call is an injection surface. A `tool_use` block cut off mid-JSON has attacker-influenceable content in a partially parsed structure. Refusing to execute tools from a `max_tokens` response is a security control, not only a correctness one.

Tool results are untrusted input. Chapter 25's third sense of validation — authority — applies here: a `tool_result` you return, and a server tool result the API inserts, both re-enter the model's context as content. Chapter 58 treats this as prompt injection; the response-handling rule is that nothing arriving in

`content` gains authority by having been in a response.

The serving model is a security-relevant fact. Under fallback the response's `model` may not be the one your policy approved. Log it, and check it if your compliance posture names specific models.

`stop_details.explanation` **is prose, not a contract.** Chapter 25 recorded that *"the text is not stable, so display it rather than parse it"* — and displaying it means displaying attacker-adjacent text to a user, so escape it like any other untrusted string.

An empty response is not a safe default. The one-liner's `""` is indistinguishable from a refusal, and a downstream system that treats empty as "nothing to do" will silently skip exactly the requests a safety classifier stopped.

Never log whole responses at debug level without redaction. `content` carries thinking blocks, tool inputs and user data; Chapter 26's leak scanner exists because logs outlive the incidents that created them.

28.24 Real Production Examples

A support tool that answered nothing. A team replaced `content[0].text` with the documentation's one-liner after a run of `IndexError`s. The crashes stopped and a slow trickle of empty replies began, at a rate too low to notice and too steady to explain. They were refusals: HTTP 200, empty `content`, `stop_reason: "refusal"`, and an extractor that returned `""` for all of them. The fix was a reading object with a `FALLBACK` action.

An agent loop that lost a search. After enabling web search, a loop began receiving result blocks for calls it had no record of. The cause was the mixed `tool_use` response: Claude had called `web_search` and a client tool together, the loop had answered its own tool and ignored the `server_tool_use` block, and the API ran the search on the continuation. Nothing was broken; the loop's model of the conversation was.

A 400 that named a tool nobody called. A developer added a friendly `"Here are the results!"` text block after the `tool_result` blocks. The error named `web_search` and an id from the previous response. The text had ended the assistant turn and stranded the waiting server tool.

A cost model that was consistently 45 per cent low. A finance dashboard summed `input_tokens` and `output_tokens` from stored responses. Once prompt caching was enabled the model stopped tracking the invoice, because two of the four billable token fields were never being read.

28.25 Certification Notes

Concept	Domain	Weight	What is examinable
The `Message` object	D4 Prompt Engineering & Structured Output	20%	every field; `model` is the **serving** model
`content` as typed blocks	D4	20%	no guaranteed first block; may be empty; may be several

Concept	Domain	Weight	What is examinable
The seven stop reasons	D4	20%	which two are non-terminal, which two are complete
Stop reasons vs errors	D5 Context Management & Reliability	15%	a stop reason is a successful response
`stop_details`	D4	20%	**null for every stop reason but** `refusal`
`pause_turn`	D1 Agentic Architecture & Orchestration	27%	server loop only; default **10 iterations**; send content back
Unresolved server tools	D1	27%	**no marker**; detect by id; runs on the continuation
The continuation contract	D1	27%	`tool_result` blocks only; same `tools` array
`model_context_window_exceeded`	D5	15%	accepted on 4.5+; typed only in the SDK **beta** namespace
`usage`	D5	15%	cache reads draw no ITPM; thinking is a split of output
Streaming	D2 Tool Design & MCP Integration	18%	`stop_reason` null in `message_start`, set in `message_delta`

CERTIFICATION NOTE

Exam tips

Highest yield: **the two non-terminal stop reasons are** `tool_use` **and** `pause_turn`, and they want opposite continuations. `tool_use` → a user message of nothing but `tool_result` blocks. `pause_turn` → the assistant content, sent back unchanged.

`pause_turn` **is the server's loop, never yours**, with a default of **10 iterations per request**. If a question describes waiting on *your* tool, the answer is `tool_use`.

`stop_details` **is null for every stop reason except** `refusal` — and can be null there too.

Only `end_turn` **and** `stop_sequence` mean the response is complete. `max_tokens` and `model_context_window_exceeded` are terminal and truncated.

If a question describes a `tool_use` response containing a `server_tool_use` block with no result, the answer is that **the server tool has not run** and there is **no other marker** — you detect it by id.

If a continuation is rejected, check three things in order: nothing but `tool_result` blocks, one per `tool_use` id, and the **same** `tools` **array**.

And remember `model_context_window_exceeded` is **typed only in the SDKs' beta namespace**.

28.26 Twenty Practice Questions

1. List every field of the `Message` object.
2. What does the top-level `model` field name?

3. Which stop reasons are non-terminal, and what does each want back?
4. Which stop reasons mean the response is complete?
5. When is `stop_details` non-null?
6. Can `content` be empty on a successful, complete response?
7. Why is `content[0].text` wrong?
8. What is wrong with `next((b.text for b in content if b.type == "text"), "")`?
9. What is the default `pause_turn` iteration limit, and whose loop is it?
10. How do you detect a server tool call that has not run?
11. What are the two extra rules for the `tool_use` continuation message?
12. What happens if you append text after the `tool_result` blocks?
13. What error do you get if you drop the waiting server tool from `tools`?
14. Why can a `max_tokens` response contain a tool call you must not execute?
15. Where does `stop_reason` appear when streaming?
16. What is special about the typing of `model_context_window_exceeded`?
17. Which `usage` field draws nothing from ITPM?
18. Is `thinking_tokens` inside or in addition to `output_tokens`?
19. What is `usage.iterations` for?
20. Why must a reader raise on an unknown `stop_reason`?

28.27 Answers and Explanations

1. `id`, `type`, `role`, `content`, `model`, `stop_reason`, `stop_sequence`, `stop_details`, `usage`. Omitting `stop_details` is the common gap.
2. **The model that served the request** — which under fallback is not the one you asked for.
3. `tool_use` wants a user message of `tool_result` blocks; `pause_turn` wants the assistant content back unchanged. Swapping them is a 400 both ways.
4. `end_turn` **and** `stop_sequence` **only.** `max_tokens` and `model_context_window_exceeded` are terminal but truncated; naming them complete is the distractor.
5. **Only on a refusal** — and even then it may be null when the decline maps to no named category.
6. **Yes.** `end_turn` with empty `content` is documented, as is an empty `content` on a refusal.
7. There is **no guaranteed first block** — it can be `thinking`, `tool_use` or `fallback` — and `content` can be empty. Panel A: correct on 6 of 18.
8. **It never raises.** It returns `""` for a refusal, for an empty answer and for unrecognised content, and it discards every text block after the first. Silently wrong on 3 of 18.
9. **10 iterations per request**, and it is the **server's** sampling loop while running server tools. A client tool waiting on you is `tool_use`, never `pause_turn`.
10. **Match each** `server_tool_use` **or** `mcp_tool_use` `id` **against the result blocks.** *"There is no other marker for the state."*
11. **Nothing except the** `tool_result` **blocks, and the same** `tools` **array.**

12. **It ends the assistant turn**, which strands any waiting server tool and produces a 400 naming that tool's id.
13. A 400 ending `but no web_search tool was provided`.
14. Generation stopped mid-JSON, so the `tool_use` block exists with truncated input.
15. `null` **in** `message_start`**, set in** `message_delta`**, absent from every other event.**
16. It is **typed only in the SDKs' beta namespace** — call `client.beta.messages`. On Sonnet 4.5 and newer the API returns it with no beta header; earlier models need `model-context-window-exceeded-2025-08-26`.
17. `cache_read_input_tokens`**.** It occupies a context seat and bills at 0.1×, but draws nothing from ITPM.
18. **Inside.** It is a split of `output_tokens`; adding them double-counts.
19. Under server-side fallback it **lists every attempt**, declined ones as `message` and the server's as `fallback_message` (Chapter 25).
20. Because the versioning contract explicitly permits **new variants of enum-like output values**, so an `else` branch will one day treat an unknown reason as a normal completion.

28.28 Ten Practical Exercises

1. Grep your codebase for `content[0]` and delete every occurrence.
2. Grep for `if block.type == "text"` inside a `next(...)` and check what happens when it finds nothing.
3. Add a counter for each of the seven stop reasons and look at the shape after a day.
4. Assert `thinking_tokens <= output_tokens` on every response for a week.
5. Compare your cost model against one real invoice line, including cache fields.
6. Log the response's `model` alongside the requested one and count the differences.
7. Feed an invented `stop_reason` through your handler and see what it does.
8. Feed a response whose `content` is `[]` through your handler.
9. Build the mixed `tool_use` fixture from §28.4 and check whether your loop notices the server tool.
10. Take one `max_tokens` response and confirm your code refuses to execute its tool call.

28.29 Five Challenge Labs

1. **The equivalence harness.** Extend Chapter 17's SSE reassembler and assert that the streamed and non-streamed readings of all eighteen fixtures are identical objects, not merely identical strings.
2. **The loop that never loses a tool.** Build an agent loop that tracks every `tool_use`, `server_tool_use` and `mcp_tool_use` id from the moment it appears until its result arrives, and fails loudly on any id that is still open when the turn goes terminal.
3. **The truncation continuer.** Implement the documentation's "ensuring complete responses" pattern, then measure how often the continuation duplicates or drops a sentence at the join.

4. **The usage reconciler.** Sum `usage` across a day of stored responses and reconcile it against the Usage and Cost API from Chapter 19, publishing every discrepancy.
5. **The prefill tension, resolved.** Design the smallest experiment that would settle §28.5's UNKNOWN — whether a `pause_turn` continuation counts as an assistant prefill on 4.6 and later — and say what each outcome would imply.

28.30 Ten Interview Questions

1. Walk me through everything you check on a Messages API response, in order.
2. Why is `stop_reason` read before `content`?
3. A response has `stop_reason: "end_turn"` and empty `content`. What happened, and what do you do?
4. What is the difference between terminal and complete?
5. How do you know a server tool call has not run?
6. A continuation returns a 400 naming a tool your code never called. Explain.
7. When is a stop reason an error?
8. Why might your cost model be 45 per cent low?
9. How would you handle a `stop_reason` your code has never seen?
10. What changes about response handling when you stream?

28.31 Summary

Chapter 16 taught the request and named a bug. This chapter taught the response and retired it.

The `Message` object has nine fields and two of them are quietly load-bearing. `model` is the **serving** model, which under Chapter 25's fallback is not necessarily the one you asked for, so cost attribution and evaluation both depend on reading it rather than assuming it. And `stop_details` is **null for every stop reason except** `refusal`, and can be null there too — which makes it useless as a discriminator and leaves exactly one field fit for the job. Branch on `stop_reason`, before anything else, always.

`content` is an array of typed blocks, and every property that makes it awkward is deliberate. There is no guaranteed first block: a thinking block, a `fallback` block or a tool call can occupy index 0. There may be several text blocks, and the answer is their concatenation. The array may be **empty** on a successful, complete response. And new block types will appear, because Chapter 26's versioning contract reserves the right to add them. Those four facts are the complete explanation of four chapters' worth of `content[0].text` failures, and the fix is one reader, in one place, that no call site can bypass.

The measurement that gives this chapter its argument is not about the naive reader at all. Three readers over eighteen fixtures: `content[0].text` is correct on **6** and raises on **10**; the total reader is correct on **16** and raises on the **2** it should; and the documentation's own recommended one-liner is correct on **15**, raises on **0**, and is **silently wrong on 3**. It cannot distinguish a refusal from an empty answer from a response made of blocks it does not recognise, because all three become `""`. It is a strictly better line of code than the one it replaces and a strictly worse handler than it appears to be, and the distinction that matters is not correct-versus-incorrect but **loud versus silent**.

Beyond the reader, three contracts. There is a response state with **no `stop_reason` of its own** — a `tool_use` response carrying an unrun `server_tool_use` block — and the documentation says outright that *"there is no other marker"*; a `stop_reason`-only check finds **1 of 2** such fixtures, and the rest must be computed by matching identifiers. The two non-terminal stop reasons want **opposite** continuations, and the eight documented 400s around them were reproduced offline, along with one genuine tension the documentation does not resolve — the `pause_turn` recipe ends the message array on an assistant message, which the prefill rule forbids on 4.6 and later — recorded as **UNKNOWN** and raising rather than guessed at. And `usage` reports **three different numbers**: one response occupying 206,000 context seats drew 18,000 against ITPM and cost **$0.40**, against **$0.21** read naively — **47.50 per cent** understated — with **40.0 per cent** of the real bill being thinking tokens that nobody will ever read.

KEY TAKEAWAYS

1. `stop_reason` **is the only field that tells you what the content means.** Read it first; `stop_details` cannot do the job.
2. `model` **is the serving model.** Log it, not the one you requested.
3. **Five stop reasons are terminal; only** `end_turn` **and** `stop_sequence` **are complete.** Label or continue everything else.
4. `content` **has no guaranteed first block, may hold several text blocks, and may be empty.** Join; never index.
5. **A reader that raises has told you something; one that returns** `""` **has not.** 6 correct / 10 raised, 15 correct / 3 silent, 16 correct / 2 raised.
6. **New stop reasons and block types are permitted by the versioning contract.** Raise on both.
7. **An unresolved server tool call has no marker.** Match `server_tool_use` ids against result blocks; it runs on your continuation.
8. **The two non-terminal reasons want opposite things**, and eight documented 400s surround the near misses.
9. **Never execute a tool from a** `max_tokens` **response** — the block can be cut off mid-JSON.
10. `usage` **is three numbers.** Cache reads draw no ITPM, thinking is a split of output, and reading two fields understated one bill by 47.50 per cent.

28.32 Glossary

Term	Definition
Complete	The content is the whole of what Claude meant to say — `end_turn` and `stop_sequence` only.
`content`	The array of typed blocks a response carries; never a string.
Content block	One typed element of `content`: `text`, `thinking`, `tool_use`, `fallback` and others.
`fallback` block	Marks a server-side handoff from one model to another (Chapter 25).
`mcp_tool_use`	A tool call made over the Model Context Protocol (Chapters 36–40).

Term	Definition
`Message`	The object every successful Messages API call returns.
`model_context_window_exceeded`	A stop reason meaning the window filled; typed only in the SDKs' beta namespace.
`pause_turn`	A non-terminal stop reason: the **server's** tool loop hit its 10-iteration limit.
`server_tool_use`	A call to a server-side tool such as web search; unresolved when no result block matches its id.
`stop_details`	Refusal metadata; **null for every other stop reason**.
`stop_reason`	The control-flow branch of a successful response; seven documented values.
Terminal	The assistant turn is over and the API is waiting for nothing — five of the seven values.
`tool_use`	A non-terminal stop reason: Claude is calling one of **your** tools.
Total function	A function defined for every input — desirable in mathematics, dangerous in a response reader.
`usage`	The token accounting object: context seats, rate-limit draw and the bill, in one place.
`usage.iterations`	Every attempt made under server-side fallback (Chapter 25).

28.33 Further Reading

Primary sources, verified on 1 August 2026 at `platform.claude.com/docs`:

- `build-with-claude/handling-stop-reasons` — the quick-reference table, all seven values, the unresolved-server-tool detection rule, the four continuation 400s, stop reasons versus errors, and the streaming rule. The single most important page for this chapter.
- `build-with-claude/refusals-and-fallback` — `stop_details`, the four categories, and the `fallback` content block (Chapter 25).
- `build-with-claude/streaming` — the event sequence and where `stop_reason` arrives (Chapter 17).
- `build-with-claude/working-with-messages` — the response shape and the statelessness rule.
- `agents-and-tools/tool-use/handle-tool-calls` — the `tool_result` continuation in full (Chapters 31–35).
- `api/errors` — the distinction between a stop reason and a 4xx.

Note that the API reference pages under `platform.claude.com/docs/en/api/...` are **client-rendered** and return no usable content to an ordinary HTTP fetch; the prose guides above carry the same material.

Where to go next in this book. Chapter 29 takes the other branch — what to do when the call fails rather than completes — turning Chapter 26's retry arithmetic and Chapter 27's 400s into a discipline. **Chapter 30** closes Part 5 with Batch, prompt caching and cost controls, where §28.16's `usage` fields become the instruments. **Chapters 31–35** build the tool-use loop this chapter's non-terminal stop reasons imply, and **Chapter 34** covers the server tools whose unresolved state §28.14 measures. Further

out, **Chapters 47–51** turn the loop into an agent.